

AgendAI Production Roadmap

Guia completo de deploy em 3 fases, mapeamento de camadas arquiteturais, 7 padrões de produção, análise de gaps agenticos, e integração máxima com serviços gerenciados AWS Bedrock e Vertex AI.

3

FASES
DE
DEPLOY

109

TESTES
AUTOMATIZADOS

17

ADRS
DOCUMENTADOS

7

PADRÕES
DE
PRODUÇÃO

8

SERVIÇOS
GERENCIADOS

Stack: LangGraph · GPT-4o-mini · Node.js · Next.js 14 · nginx · PostgreSQL (Neon) · Redis (Upstash)
Cloud: GCP (Cloud Run · Vertex AI · Memory Bank) + AWS (Bedrock Guardrails · ECS Fargate)

SUMÁRIO

01	Deployment Roadmap — duas fases	3
	Fase 1: Render + GitHub Actions	3
	Fase 2: Terraform + Cloud (GCP/AWS)	3
02	CI/CD Pipeline — testes como gate de deploy	4
03	8 Camadas de Arquitetura	5
04	7 Design Patterns de Produção	6
05	Análise de Gaps Agenticos	7
06	Prioridades de Implementação	8
07	Fase 3 — Integração Máxima com Serviços Gerenciados	10
	Evolução por componente: Fase 1 → 2 → 3	10
	AWS Bedrock Guardrails	11
	Vertex AI Memory Bank + Agent Engine Sessions	11
	Amazon Cognito + Vertex AI Evaluation	12
	Arquitetura completa Fase 3 + estimativa de custo	12
08	Redis no LangGraph Server — dois papéis distintos	14
	Redis de cache (API Node.js) vs Redis de streaming (LangGraph)	14
	Stack gratuito completo — Opção B com LangGraph Server	14

01 — DEPLOYMENT ROADMAP

Duas fases de deploy

Fase 1 coloca o projeto no ar rapidamente com URL pública e CI/CD real. Fase 2 eleva para infraestrutura cloud production-grade com IaC (Terraform). A Fase 3 (seção 07) integra serviços gerenciados AWS e GCP sobre essa base.

► FASE 1 — AGORA · semana 1-2

Render + GitHub Actions

Custo estimado: ~\$7-15/mês

01. Migrar SQLite → Neon PostgreSQL (free tier)
02. Publicar 4 serviços: nginx público, api + agent como Private Services, agent-ui-pro público
03. GitHub Actions: tests gate → build → deploy
04. Mover secrets para GitHub Secrets + Render env vars
05. Badge CI verde no README + URL de produção
06. LangSmith já integrado — screenshot do dashboard de traces para portfólio

◎ FASE 2 — CLOUD IaC · semana 3-5

Terraform + GCP / AWS

Pode destruir após screenshot para portfólio

01. Criar /infra/terraform no repositório
02. GCP: Cloud Run × 3 + Cloud LB + URL Map + Cloud SQL + Memorystore
03. AWS: ECS Fargate + ALB + RDS PostgreSQL + ElastiCache
04. Secret Manager / Secrets Manager (substituir .env)
05. Cloud Armor / AWS WAF para rate limiting gerenciado
06. terraform apply → screenshot → terraform destroy

Estrutura de repositório alvo

```
AgendAI/
├─ agent/           # Python · LangGraph
├─ api/             # Node.js · Express
├─ agent-ui-pro/    # Next.js 14
├─ nginx/           # Reverse proxy
├─ infra/           # NOVO — infraestrutura como código
│   └─ terraform/
│       └─ gcp/      # Cloud Run + LB + Cloud SQL + Memorystore
│       └─ aws/      # ECS Fargate + ALB + RDS + ElastiCache
│       └─ render/   # render.yaml — deploy config
├─ .github/workflows/ # NOVO — CI/CD
│   └─ ci.yml        # PR: run tests (39 Jest + 70 pytest)
│   └─ deploy.yml    # main: build + deploy to Render
```

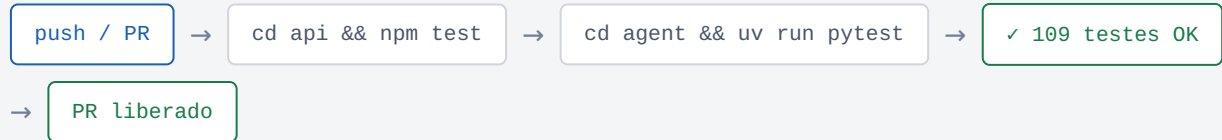
- └─ docker-compose.yml
- └─ README.md

02 – GITHUB ACTIONS PIPELINE

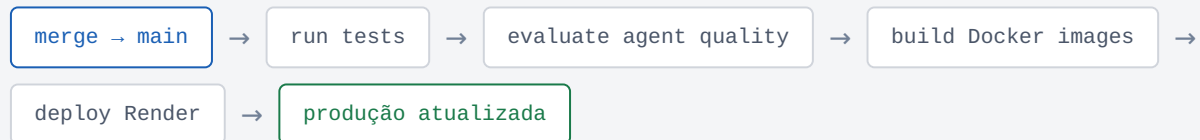
CI/CD: testes como gate de deploy

Nenhum código chega em produção sem os 109 testes passarem. Dois workflows separados: CI em PRs, CD no merge para main.

ci.yml – pull request



deploy.yml – merge para main



```

# .github/workflows/ci.yml (esqueleto)
name: CI
on: [push, pull_request]
jobs:
  test-api:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: cd api && npm ci && npm test

  test-agent:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v3
      - run: cd agent && uv run pytest --tb=short
  
```

► INTERVIEW SIGNAL

"Tenho dois workflows separados: CI em todo PR que bloqueia merge se qualquer um dos 109 testes falhar, e CD que só roda no merge para main após os testes passarem. Nenhum código chega em produção sem cobertura."

03 – CAMADAS DE ARQUITETURA

8 camadas: local → Render → Cloud

Cada camada mapeia a tecnologia atual, a solução no Render e a solução escalável em cloud (GCP/AWS).

CAMADA	LOCAL + RENDER	CLOUD (GCP/AWS)	FASE 3 (MANAGED)
01 Presentation	Next.js 14 · agent-ui-pro porta 3002 · Web Service	Cloud Run / ECS Fargate HTTPS automático · escala 0→N	Firebase Auth integrado user_id no JWT propagado
02 Gateway	nginx Web Service público api + agent como Private Services	Cloud LB + URL Map (GCP) / ALB (AWS) SSE support · sem nginx	Cloud Armor / WAF rate limiting gerenciado
03 Orchestration	LangGraph v1.0+ Python 3.11 6 nós · 5 tools · dev server	Cloud Run / ECS Fargate InMemory → PostgresSaver	Agent Engine Sessions (GA) Memory Bank (GA)
04 REST API	Node.js 20 + Express 4 rate-limit 100/15min · cache TTL	Cloud Run / ECS Fargate Cloud Armor / WAF	Bedrock Guardrails (input/output) pybreaker + CB
05 Data	SQLite → Neon PostgreSQL migração obrigatória na Fase 1	Cloud SQL / RDS PostgreSQL managed · backups automáticos	Cloud SQL / RDS sem mudança
06 Cache	node-cache in-memory TTL 60s · single-instance only	Memorystore / ElastiCache Redis distribuído · ~\$15-35/mês	Memorystore / ElastiCache + semantic cache se RAG
07 Observability	LangSmith (já existe) traces · latência · tool calls	LangSmith + Cloud Monitoring infra + agente	+ Vertex AI Evaluation quality gate no CI/CD
08 Infrastructure	render.yaml declarativo git-based deploy · auto HTTPS	Terraform /infra/ terraform/ gcp/ e aws/ no mesmo repo	Terraform + managed services apply → screenshot → destroy

04 — 7 DESIGN PATTERNS DE PRODUÇÃO

Status atual × solução gerenciada em cloud

#	PADRÃO	STATUS NO CODEBASE	SERVIÇO GERENCIADO (CLOUD)	PRIORIDADE
01	API Gateway	✓ nginx + LANGGRAPH_AUTH_TOKEN	Cloud LB + URL Map (GCP) / ALB (AWS) <i>Cloud API Gateway: streaming NÃO suportado (doc. oficial)</i>	fase 2
02	Rate Limiting	✓ express-rate-limit 100/15min por IP	Cloud Armor (GCP) / AWS WAF rate rules substitui código sem alterar app	fase 2
03	Caching	parcial node-cache in-memory TTL 60s single-instance apenas	Memorystore Redis (GCP) ~\$35/mês ElastiCache Redis (AWS) ~\$15/mês	fase 2
04	Message Queues	não necessário fluxo síncrono por design	Pub/Sub / Cloud Tasks (GCP) · SQS (AWS) adicionar só se precisar async	pular
05	Circuit Breaker	parcial tenacity: email + TTS apenas gap llm_core.py sem CB	App Mesh / Cloud Service Mesh: overkill Solução: pybreaker + Bedrock Guardrails	P1 urgente
06	Load Balancing	✓ nginx proxy local	Automático: Cloud Run / ECS Fargate não precisa configurar	grátis
07	Auto Scaling	✓ healthchecks no docker-compose	Cloud Run: escala 0→N automático ECS: min/max tasks no Terraform	grátis

Nota sobre caching semântico: O cache atual (node-cache TTL 60s) é o tipo correto para o AgendAI — cacheia resultados de queries SQL no endpoint /horarios/disponiveis. Caching semântico (baseado em embeddings) só faria sentido se o projeto adicionasse uma base de conhecimento / FAQ com RAG. Sem RAG, não há chamadas a embedding APIs para cachear.

O que falta para ser production-grade

Comparando o codebase atual com os requisitos de uma aplicação agêntica production-grade, baseado nos whitepapers *Prototype to Production*, *Context Engineering: Sessions & Memory* e *Agentic Design Patterns*.

✓ O que AgendAI já tem	✗ O que está faltando
✓ LangGraph com grafo de 6 nós e roteamento condicional	✗ Memória por usuário — InMemoryCheckpointner (ADR-014) reseta em restart. Sem sessão persistente por user_id.
✓ Function calling com 5 tools (GPT-4o-mini)	✗ Auth de usuário — só token de serviço (nginx). Sem identidade de usuário, JWT, ou sessão individual.
✓ LangSmith — traces, latência por nó, tool calls	✗ Guardrails — sem validação de input (prompt injection), sem filtragem de output, sem bloqueio off-scope.
✓ Rate limiting por IP (express-rate-limit)	✗ CB no LLM — tenacity cobre email + TTS mas llm_core.py não tem retry nem CB para RateLimitError.
✓ Caching com TTL + invalidação em escrita	✗ Sessão persistente — conversas não sobrevivem a restarts. Necessário para multi-turn context.
✓ Retry com tenacity (email + TTS)	✗ Logs estruturados — sem correlation IDs, sem JSON structured logging, sem request tracing.
✓ STT (Whisper) + TTS (OpenAI) — entrada multimodal	✗ PII handling — dados de pacientes passam pelo agente sem redação explícita.
✓ nginx como gateway com token de auth de serviço	✗ Métricas + alertas — LangSmith monitora o agente mas sem alertas de latência ou custo de tokens.
✓ Healthchecks no docker-compose	~ Cache distribuído — node-cache não sincroniza em múltiplas instâncias.
✓ 109 testes automatizados (39 Jest + 70 pytest)	
✓ 17 ADRs documentando decisões arquiteturais	
✓ Stack poliglota Node.js + Python com camadas separadas	

► CONTEXTO — PROTOTYPE TO PRODUCTION (GOOGLE, 2025)

"Skipping these final steps could cause several problems. A user discovers they can access a confidential internal database through your agent because authentication was improperly configured. An agent generates a large consumption bill over the weekend, but no one knows why because you didn't set up any monitoring."

06 — PRIORIDADES DE IMPLEMENTAÇÃO

O que adicionar e em que ordem

Ranqueado por impacto técnico × esforço. Os 5 primeiros transformam o projeto de demo para agêntico real.

P1

Circuit Breaker no llm_core.py

30 min

Adicionar pybreaker nas chamadas OpenAI. RateLimitError, APITimeoutError, InternalServerError devem ativar o circuito. É a diferença entre retry (o que tenacity faz) e fail-fast quando a API está fora do ar.

```
from pybreaker import CircuitBreaker
llm_breaker = CircuitBreaker(fail_max=3, reset_timeout=30)

@llm_breaker
def call_openai(messages, tools): ...
```

urgente

~10 linhas

impacto alto

P2

Sessão persistente por usuário (PostgresSaver ou Agent Engine Sessions)

2h

ADR-014 documenta o InMemoryCheckpoint. Substituir por PostgresSaver (Fase 1-2) ou Agent Engine Sessions (Fase 3). Cada conversa ganha um thread_id por user_id e sobrevive a restarts.

```
from langgraph.checkpoint.postgres import PostgresSaver
checkpointer = PostgresSaver.from_conn_string(DATABASE_URL)
graph = workflow.compile(checkpointer=checkpointer)
```

alta prioridade

~2h

habilita per-user memory

P3

Autenticação de usuário básica

1 dia

Clerk (free tier), Auth0, ou Amazon Cognito. O user_id autenticado passa a ser o thread_id do LangGraph checkpointer — conecta sessão, memória e auditoria. O nginx/gateway valida o JWT antes de chegar no agente.

alta prioridade

Clerk free: ~1 dia

habilita per-user session

P4

Guardrails básicos (input validation)

4h

Novo nó LangGraph antes do `chat_with_llm` verificando: (a) prompt injection patterns, (b) tópicos off-scope para a clínica, (c) PII no input. Fase 3 substitui por Bedrock Guardrails (managed, sem código adicional).

```
def validate_input(state: AgentState):
    if is_injection(state.input) or is_off_scope(state.input):
        return {"blocked": True, "reason": "guardrail"}
    return state
```

média prioridade

~4h

requisito p/ prod real

P5

Logs estruturados + correlation IDs

3h

Middleware no Express gerando `request_id` (UUID) por request propagado nos logs. No agente Python: structlog com output JSON. Ligar `request_id` ao `trace_id` do LangSmith fecha o loop de observabilidade end-to-end.

baixa prioridade

~3h

observabilidade end-to-end

► NARRATIVE PARA ENTREVISTA – VISION COMPLETA

"O AgendAI hoje tem os fundamentos sólidos: LangGraph orquestrando o agente, LangSmith para observabilidade de traces, rate limiting, caching e retry em partes críticas. O que falta para ser produção-real são as 4 camadas que o Prototype to Production chama de 'last mile': persistência de sessão por usuário (migrando `InMemoryCheckpoint` para `PostgresSaver` ou `Agent Engine Sessions`), autenticação de identidade do usuário, guardrails de input (substituídos por Bedrock Guardrails na Fase 3), e circuit breaker nas chamadas LLM."

07 – FASE 3: INTEGRAÇÃO MÁXIMA COM SERVIÇOS GERENCIADOS

Substituir código manual por AWS Bedrock + Vertex AI

Fase 3 não é sobre trocar o framework — LangGraph e GPT-4o-mini permanecem inalterados. É sobre delegar infraestrutura de segurança, memória e observabilidade para serviços especializados usando uma abordagem **multi-cloud deliberada**: melhores serviços de cada provider.

Descoberta-chave: Bedrock Guardrails funciona com *qualquer* LLM — incluindo GPT-4o-mini — via ApplyGuardrail API. Você não precisa migrar para Claude ou Gemini. Vertex AI Memory Bank (GA jul/2025) funciona com LangGraph via chamadas diretas de API.

Evolução por componente: Fase 1 → 2 → 3

COMPONENTE	FASE 1 – RENDER	FASE 2 – TERRAFORM + CLOUD	FASE 3 – MANAGED MAX	STARS
Guardrails	pybreaker (CB apenas)	pybreaker + nó LangGraph manual	AWS Bedrock Guardrails funciona com GPT-4o-mini	★★★★★
Sessão (short-term)	InMemoryCheckpointner reset em restart	PostgresSaver (LangGraph)	Vertex AI Agent Engine Sessions (GA) managed, HIPAA compliant	★★★★☆
Memória (long-term)	não existe	não existe	Vertex AI Memory Bank (GA) extração semântica via Gemini	★★★★★
Rate limiting	express-rate-limit (código)	express-rate-limit + cloud infra	Cloud Armor / AWS WAF sem alterar código do app	★★★★☆
Autenticação	Clerk / JWT manual	Clerk / JWT manual	Amazon Cognito / Firebase Auth free tier até 50k MAU	★★★★☆
Observabilidade LLM	LangSmith (já existe)	LangSmith + Cloud Monitoring	+ Vertex AI Evaluation quality gate no CI/CD	★★★★☆
Knowledge Base	não existe	não existe	Bedrock Knowledge Bases (AWS) se adicionar FAQ/RAG → ativa semantic cache	★★★☆☆

07 – FASE 3 (CONTINUAÇÃO)

Integrações por serviço – como implementar

AWS Bedrock Guardrails – input + output filtering para GPT-4o-mini

```
# agent/nodes/llm_core.py – integração Bedrock Guardrails
import boto3
bedrock = boto3.client('bedrock-runtime', region_name='us-east-1')

def check_guardrail(text: str, source: str) -> bool:
    resp = bedrock.apply_guardrail(
        guardrailIdentifier=GUARDRAIL_ID,
        guardrailVersion='DRAFT',
        source=source,          # 'INPUT' ou 'OUTPUT'
        content=[{'text': {'text': text}}]
    )
    return resp['action'] != 'GUARDRAIL_INTERVENED'

def chat_with_llm(state: AgentState):
    # Bloqueia ANTES de chamar OpenAI – economiza tokens
    if not check_guardrail(state.input, 'INPUT'):
        return {"response": "Não posso responder essa pergunta."}
    response = openai_client.chat.completions.create(...)
    if not check_guardrail(response.content, 'OUTPUT'):
        return {"response": "Resposta não disponível."}
    return {"response": response.content}
```

Configuração no console AWS: tópicos bloqueados (assuntos não médicos), filtros de PII (CPF, email de pacientes em logs), content filters. Cada política é uma checkbox — zero código adicional.

Vertex AI Memory Bank — memória de longo prazo com LangGraph

```
# agent/nodes/llm_core.py — Memory Bank com LangGraph
from vertexai.preview.memory import MemoryBankServiceClient

memory = MemoryBankServiceClient()

def retrieve_user_context(user_id: str, query: str) -> str:
    # Busca memórias relevantes antes de cada turno
    results = memory.retrieve_memories(
        agent_engine_id=RUNTIME_ID, user_id=user_id, query=query
    )
    return "\n".join([m.content for m in results.memories])

def save_session_to_memory(user_id: str, session_events: list):
    # Roda APÓS a resposta — não bloqueia o usuário
    memory.generate_memories(
        agent_engine_id=RUNTIME_ID, user_id=user_id, events=session_events
    )

# Injetar no system_prompt antes de chamar o LLM:
user_context = retrieve_user_context(state.user_id, state.input)
system_prompt = f"{BASE_PROMPT}\n\nContexto do paciente:\n{user_context}"
```

Vertex AI Agent Engine Sessions — substitui InMemoryCheckpoint (ADR-014)

```
# Uma linha substitui InMemoryCheckpoint com isolamento por user_id
from vertexai.preview.reasoning_engines import AgentEngineCheckpoint

checkpointer = AgentEngineCheckpoint(
    agent_engine_id=RUNTIME_ID,
    user_id=state.user_id # isolamento automático por usuário
)
graph = workflow.compile(checkpointer=checkpointer)
```

Vertex AI Evaluation — quality gate automático no CI/CD

```
# deploy.yml — quality gate duplo: testes + qualidade do agente
- name: Run unit tests
  run: cd api && npm test && cd ../agent && uv run pytest

- name: Evaluate agent quality (Vertex AI)
  run: |
    python scripts/evaluate_agent.py \
      --dataset golden_set.jsonl \
      --threshold tool_call_success=0.85 \
      --threshold scope_adherence=0.90

- name: Deploy to production    # só roda se ambos passarem
  if: success()
```

07 — FASE 3 (CONTINUAÇÃO)

Arquitetura completa + estimativa de custo

Fluxo completo — Fase 3 com serviços gerenciados

```

Browser → Firebase Auth / Amazon Cognito → JWT com user_id embutido
|
Cloud LB + URL Map (GCP)      ← SSE support, HTTPS termination, único domínio
|
Cloud Armor / AWS WAF         ← rate limiting gerenciado (sem alterar código do app)
|
Cloud Run: agent-ui-pro       ← Next.js 14, porta 3002
|
Cloud Run: agent (LangGraph)
  |— retrieve_user_context() → Vertex AI Memory Bank    [long-term memory]
  |— checkpointer           → Agent Engine Sessions    [short-term session]
  |
  |— Bedrock Guardrails INPUT check      ← bloqueia antes do LLM
  |
  |— GPT-4o-mini (OpenAI)                ← LLM inalterado
  |
  |— Bedrock Guardrails OUTPUT check     ← filtra resposta
  |
  |— Tool calls → Cloud Run: api (Node.js Express)
    |— Cloud SQL PostgreSQL              ← DB gerenciado
    |— Memorystore Redis                 ← cache distribuído
  |
  |— async: save_session_to_memory()     ← extração não bloqueia resposta

Observabilidade:
LangSmith          ← traces do agente, latência por nó (já existe)
Cloud Trace        ← distributed tracing de infra
Vertex AI Evaluation ← quality gate automático no CI/CD
Cloud Monitoring   ← alertas de latência, custo de tokens, error rate

```

Estimativa de custo mensal — Fase 3

GCP managed services

- Cloud Run (3 serviços): \$5-20/mês
- Cloud SQL PostgreSQL: ~\$10/mês
- Memorystore Redis: ~\$35/mês
- Agent Engine Sessions: billing jan/2026
- Memory Bank: ~\$0.001/memória gerada
- Cloud LB + Cloud Armor: ~\$8/mês

~\$58-80/mês

AWS managed services (adiciona)

- Bedrock Guardrails: ~\$0.75/1k chamadas
- Amazon Cognito: free até 50k MAU
- Bedrock Knowledge Bases: pago por doc/query

+~\$5-15/mês

Para portfólio: terraform apply → screenshot → terraform destroy. O código no repo prova o conhecimento sem custo mensal contínuo.

► NARRATIVE PARA ENTREVISTA – ARQUITETURA MULTI-CLOUD

"Adotei uma arquitetura multi-cloud deliberada no AgendAI. O agente roda em GCP (Cloud Run + Vertex AI Agent Engine Sessions + Memory Bank), mas os guardrails são da AWS (Bedrock Guardrails), porque o ApplyGuardrail API funciona com qualquer LLM incluindo GPT-4o-mini — não exige trocar de provider. Essa é uma decisão arquitetural real usada em enterprise: best-of-breed independente de cloud. O Memory Bank substitui PostgresSaver com algo mais sofisticado: extração semântica de fatos via Gemini em background, então o histórico de que um paciente prefere sextas-feiras é preservado entre sessões sem eu implementar um pipeline de ETL manual."

08 – REDIS NO LANGGRAPH SERVER

Dois papéis do Redis – não confundir

O AgendAI usa (ou pode usar) Redis em dois contextos completamente diferentes. Entender essa distinção é importante para dimensionar o stack e responder bem em entrevista.

ASPECTO	REDIS DE CACHE – API NODE.JS	REDIS DE STREAMING – LANGGRAPH SERVER
Para quê	Cache de respostas HTTP (horários, disponibilidade). TTL 60s, invalidação em escrita.	Pub-sub broker para SSE de background runs. Cada nó do grafo publica eventos em tempo real ao cliente.
Quem usa	Sua API Node.js (node-cache hoje, Memorystore/ElastiCache na Fase 2)	O servidor langchain/langgraph-server internamente — você só configura REDIS_URI
Sem ele	Cache cai para in-memory (node-cache). Funciona, mas não sincroniza entre múltiplas instâncias.	Streaming de runs em background não funciona. Você pode chamar o agente de forma síncrona, mas perde o SSE token-a-token.
Variável de ambiente	REDIS_CACHE_URI (sua variável)	REDIS_URI (exigida pelo LangGraph Server)
Free tier	Upstash Redis: 10k cmd/dia gratuito	Upstash Redis: mesma instância ou separada
Fase	Fase 2: Memorystore (GCP) / ElastiCache (AWS)	Fase 1 já: obrigatório para LangGraph Server em produção

Podem compartilhar a mesma instância Redis? Sim — o LangGraph Server usa prefixos de chave próprios (langgraph:), então uma única instância Upstash serve os dois propósitos sem conflito. Para portfólio, use uma instância só. Para produção real, separe por segurança operacional.

Stack gratuito completo — Opção B com LangGraph Server

A Opção B usa o servidor oficial langchain/langgraph-server (Docker) com serviços externos gratuitos. Você não roda PostgreSQL ou Redis em container — aponta para serviços managed com free tier.

Componentes e providers gratuitos

COMPONENTE	PROVIDER GRATUITO	LIMITE DO FREE TIER	VARIÁVEL DE AMBIENTE
LangGraph Server	Docker image: langchain/langgraph-server	Developer plan: 100k nodes/mês	LANGSMITH_API_KEY
PostgreSQL	Neon (neon.tech)	0.5 GB, sem cartão de crédito	DATABASE_URI
Redis (streaming + cache)	Upstash (upstash.com)	10k cmd/dia, serverless	REDIS_URI
Compute (agent)	Render Web Service	Free (dorme após 15min) ou \$7/mês Starter	—
Observabilidade	LangSmith Developer	5k traces/mês gratuito	LANGCHAIN_API_KEY

langgraph.json — arquivo de configuração obrigatório

```
# agent/langgraph.json — define o grafo para o LangGraph Server
{
  "graphs": {
    "agent": "./graph.py:graph" // nome:path do CompiledGraph
  },
  "dependencies": ["."], // instala o pacote local
  "env": ".env" // arquivo de variáveis (dev)
}
```

Fluxo de deploy — Opção B no Render

```
# 1. Buildar imagem Docker do agente
pip install langgraph-cli
langgraph build -t agendai-agent

# 2. Push para Docker Hub / GitHub Container Registry
docker tag agendai-agent ghcr.io/rc-ventura/agendai-agent:latest
docker push ghcr.io/rc-ventura/agendai-agent:latest

# 3. render.yaml – Web Service apontando para a imagem
services:
  - type: web
    name: agendai-agent
    image:
      url: ghcr.io/rc-ventura/agendai-agent:latest
    envVars:
      - key: LANGSMITH_API_KEY          # license key Developer plan
        sync: false
      - key: DATABASE_URI              # Neon PostgreSQL
        sync: false
      - key: REDIS_URI                 # Upstash Redis (streaming SSE)
        sync: false
      - key: OPENAI_API_KEY
        sync: false
      - key: LANGCHAIN_API_KEY         # LangSmith traces
        sync: false
```

Por que langgraph dev não serve para produção

FEATURE	LANGGRAPH DEV	LANGGRAPH BUILD + DOCKER
Docker	não requerido	imagem isolada
Persistência de estado	In-memory + arquivo local (perde em restart)	PostgreSQL via DATABASE_URI (Neon)
Streaming SSE	Funciona localmente	Via Redis pub-sub (Upstash)
Hot reload	sim	não – rebuild necessário
Porta padrão	2024	8123
License key obrigatória	Não	Sim — LANGSMITH_API_KEY
Para portfólio	só local/dev	deploy real no Render

► NARRATIVE PARA ENTREVISTA — REDIS NO SISTEMA

"No AgendAI o Redis tem dois papéis distintos. Na API Node.js, o Redis é cache de respostas HTTP — TTL 60s, invalidação síncrona em escrita, distribuído para suportar escala horizontal. No LangGraph Server, o Redis é um pub-sub broker exclusivamente para streaming SSE: cada nó do grafo que termina de executar publica um evento no Redis, e o cliente faz subscribe nesse canal para receber tokens em tempo real. São cargas de trabalho diferentes — uma é key-value com TTL, a outra é pub-sub efêmero. Para portfólio, compartilho uma instância Upstash; em produção real, separaria por isolamento operacional."

Referências atualizadas

- Google Cloud: *Prototype to Production* (Nov. 2025)
- Google Cloud: *Context Engineering: Sessions & Memory* (Nov. 2025)
- Google Cloud: *Introduction to Agents* (Nov. 2025)
- Antonio Gulli: *Agentic Design Patterns*
- Google Cloud: *Agent Quality* (Nov. 2025)
- AWS: *Amazon Bedrock Guardrails* — docs.aws.amazon.com/bedrock
- GCP: *Vertex AI Agent Engine Overview* — cloud.google.com/api-gateway/docs/quotas
- LangChain: *Self-host standalone servers* — docs.langchain.com/langsmith/deploy-standalone-server
- LangChain: *LangGraph CLI reference* — docs.langchain.com/langsmith/cli
- LangChain: *LangSmith Pricing* — langchain.com/pricing (Developer plan: 100k nodes/mês grátis)